Introduction

Team 7

Henry Lai, Kimi Yokoyama, Hirna Derege, Fatimah Abdullahi

Smart Healthcare and Lifestyle Prediction Dataset

Target Variable: Heart_Disease

1. Problem and Data Snapshot

About the Dataset

Our chosen dataset represents patient portfolios and profiles to simulate real-world healthcare scenarios. Diverse information about everyone is provided, allowing insight into lifestyle, medical background, and demographic factors that might contribute to the overall health of the patient. Examples of these factors include gender, alcohol usage, and the presence of a stroke in medical history. This dataset is formatted as a .csv, containing only tabular, numeric data, with some of the variables being binary. It is a singular file, not yet separated into training and testing data, with 5,000 entries or rows.

About the Problem

The goal of diagnostic medicine is to take all necessary information about a patient and come up with a medical diagnosis. "Situated knowledge," a term coined by Donna Haraway, is the idea that all knowledge comes with bias that originates from the knower. We see these biases throughout the history of medicine in the US, contributing to racial disparities in diagnosis, treatment, and patient-provider trust. Our team goal is to train a model that can calculate a health score that would be solely based on medical records, free from bias.

Target Variable

Heart_Disease

Main Input Features

Age, Gender, BMI, Exercise_Level, Smoking, Alcohol, Blood_Pressure, Cholesterol, Glucose, Fatigue, Chest_Pain, Dizziness, Diabetes, Stroke

2. Exploratory Data Analysis

Imports

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = pd.read_csv('smart_healthcare_dataset.csv')
data.head()
```

Out[ ]:

	age	gender	bmi	exercise_level	smoking	alcohol	blood_pressure	cholesterol
0	56	Male	22.6	2	1	1	169	225
1	69	Female	28.2	0	1	1	136	230
2	46	Female	25.1	1	0	1	142	221
3	32	Female	18.0	0	0	1	173	296
4	60	Female	20.1	2	1	0	130	292

Summary Statistics

In [ ]:
```
features = ['age', 'gender', 'bmi', 'exercise_level', 'smoking', 'alcohol',
            'blood_pressure', 'cholesterol', 'glucose', 'fatigue',
            'chest_pain', 'dizziness', 'diabetes', 'stroke']
summary_stat = data[features].describe()
summary_stat
```

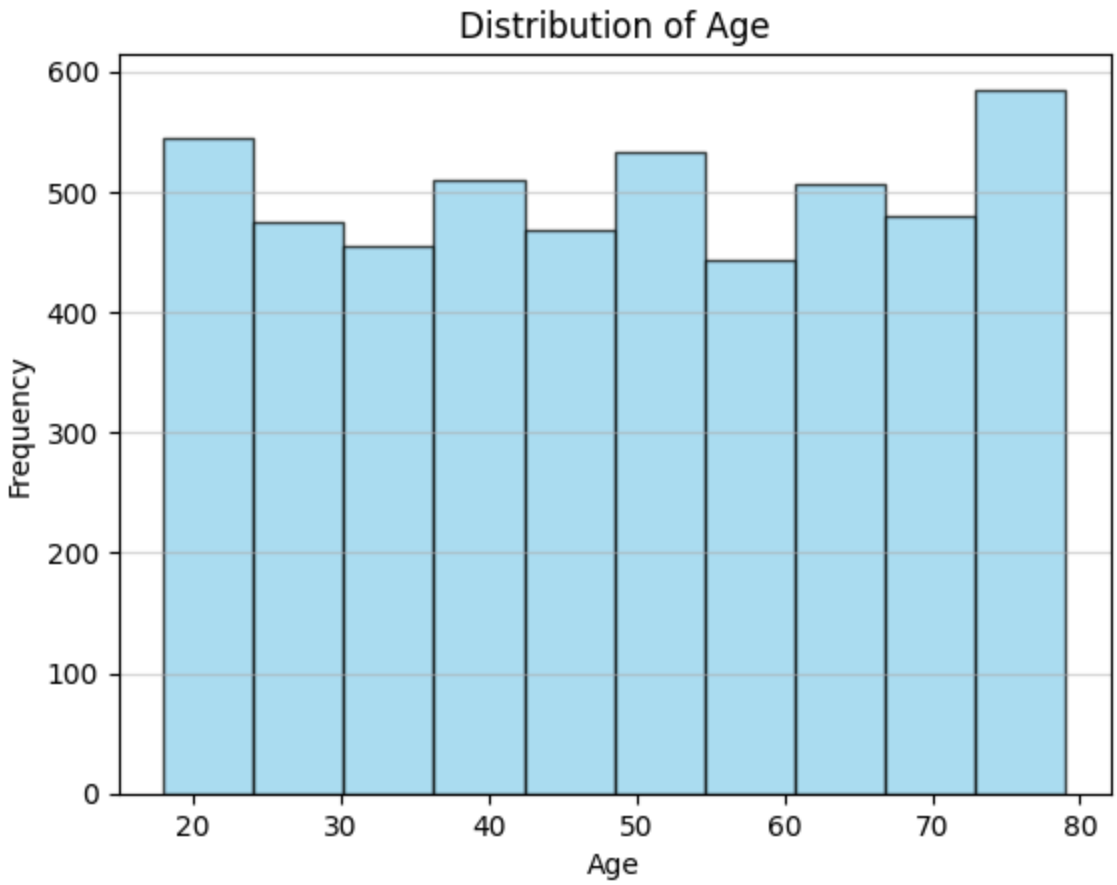
Out[ ]:

	age	bmi	exercise_level	smoking	alcohol	blood_p
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000
mean	48.805600	24.967800	1.005800	0.507600	0.501400	134
std	17.906991	4.960379	0.815291	0.499992	0.500048	26
min	18.000000	15.000000	0.000000	0.000000	0.000000	90
25%	34.000000	21.500000	0.000000	0.000000	0.000000	111
50%	49.000000	24.900000	1.000000	1.000000	1.000000	134
75%	64.000000	28.400000	2.000000	1.000000	1.000000	157
max	79.000000	42.600000	2.000000	1.000000	1.000000	179

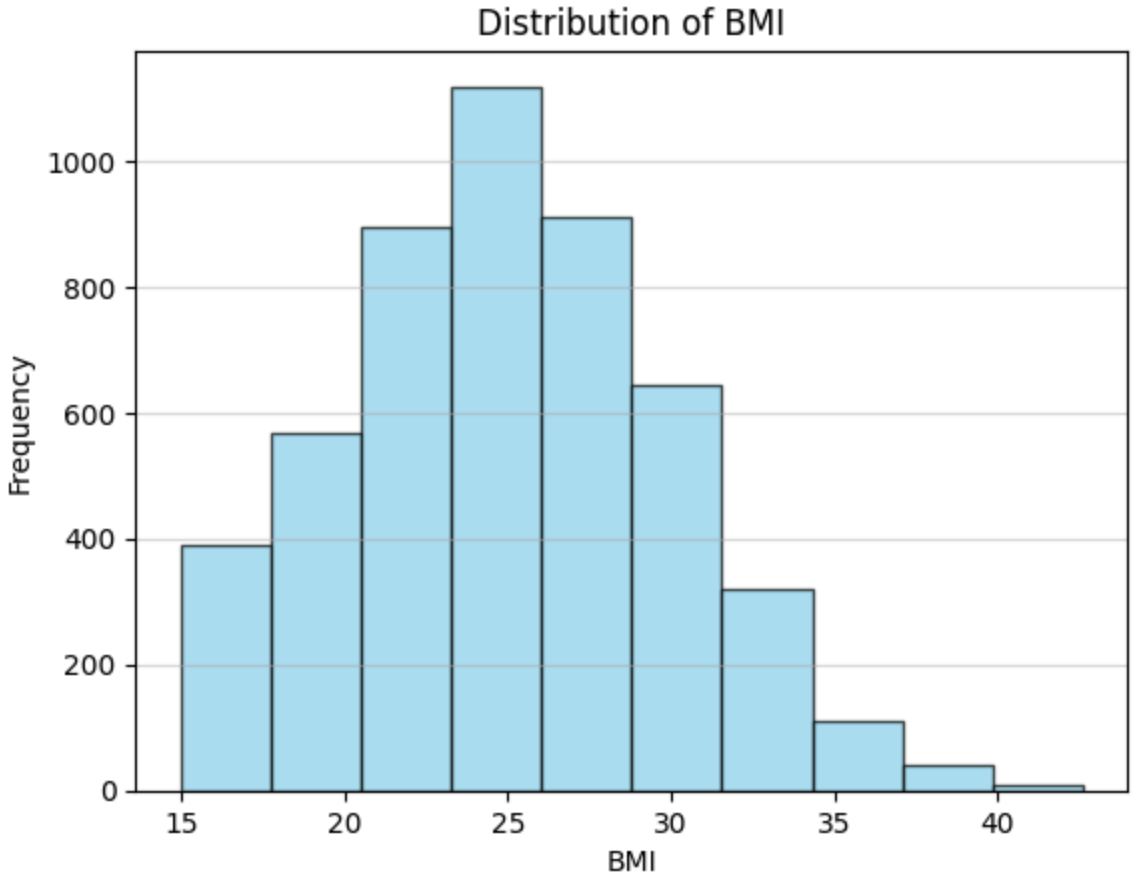
Histograms

In [ ]:
```
plt.hist(data['age'], bins=10, color='skyblue', edgecolor='black', alpha=0.7
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.title("Distribution of Age")
```

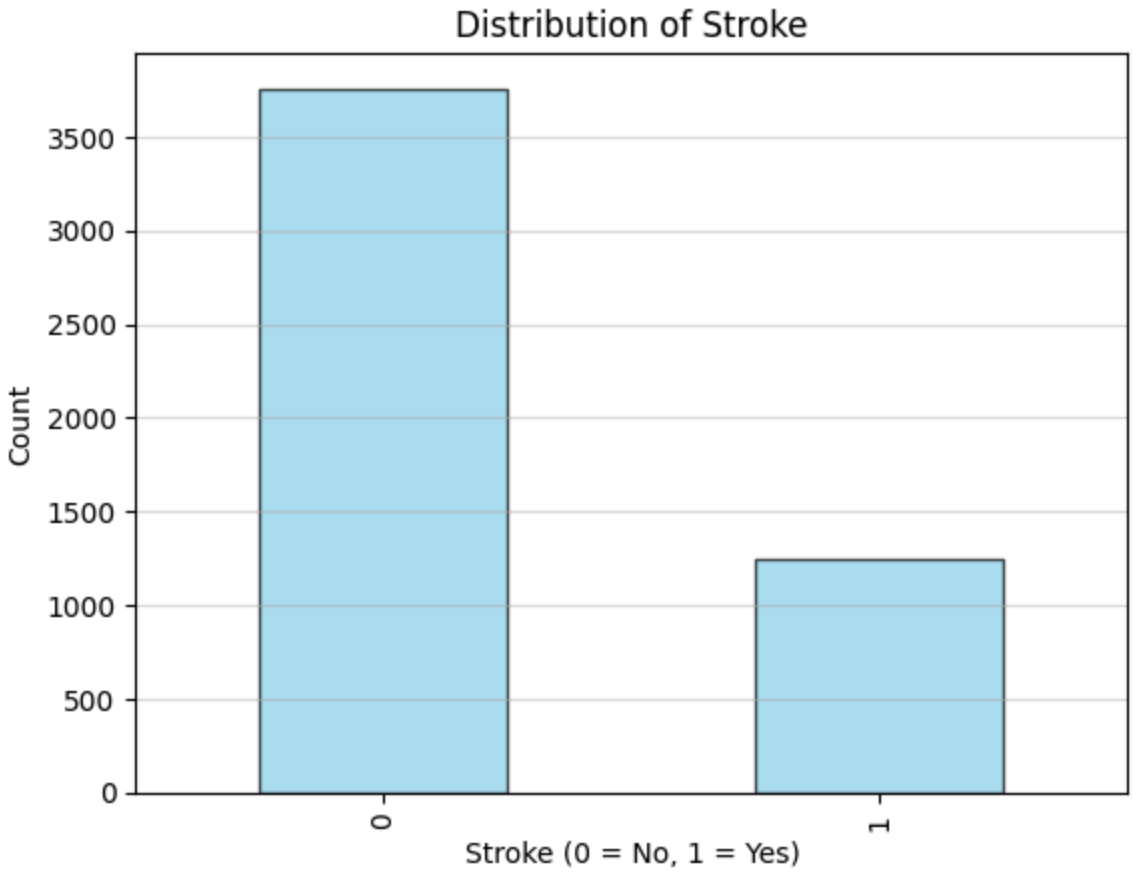
```
plt.grid(axis='y', alpha=0.5)
plt.show()
```



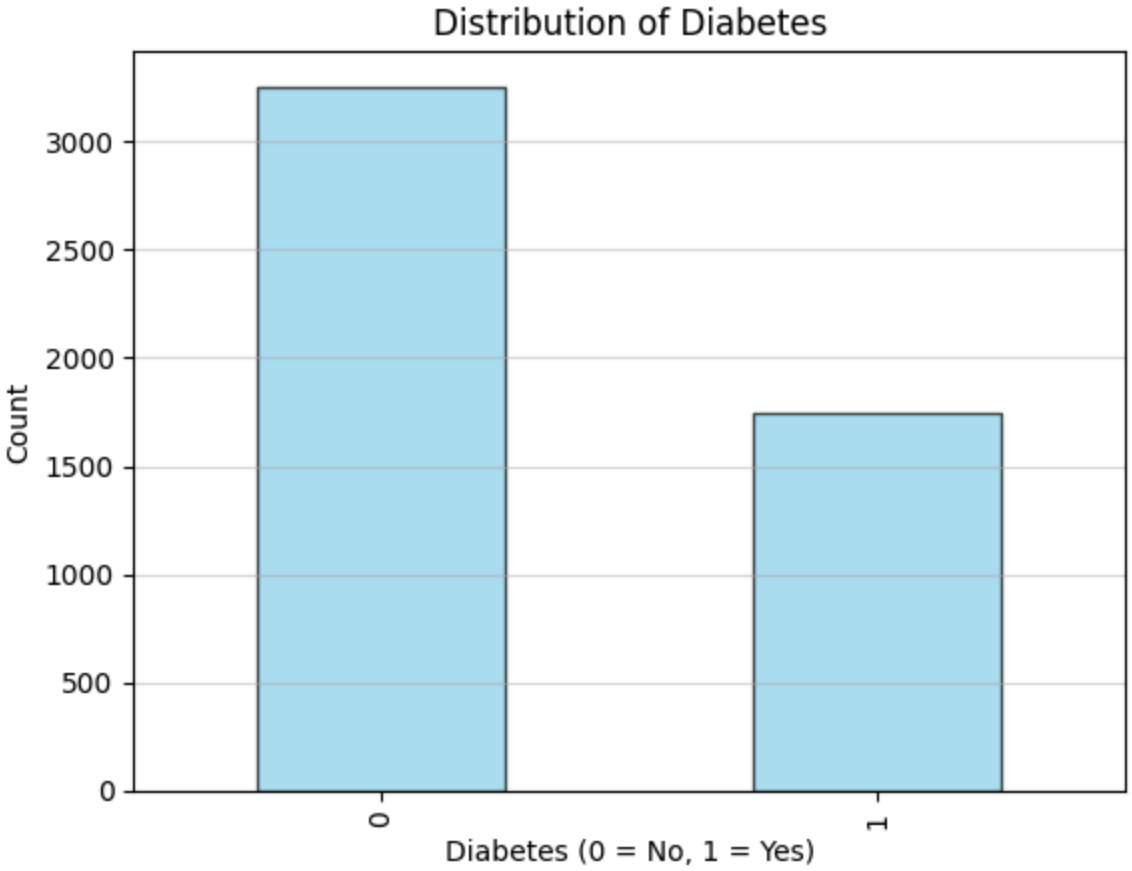
In [ ]:
```
plt.hist(data['bmi'], bins=10, color='skyblue', edgecolor='black', alpha=0.7
plt.xlabel("BMI")
plt.ylabel("Frequency")
plt.title("Distribution of BMI")
plt.grid(axis='y', alpha=0.5)
plt.show()
```



In [ ]:
```
data['stroke'].value_counts().sort_index().plot(
    kind='bar',
    color='skyblue',
    edgecolor='black',
    alpha=0.7
)
plt.xlabel("Stroke (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.title("Distribution of Stroke")
plt.grid(axis='y', alpha=0.5)
plt.show()
```



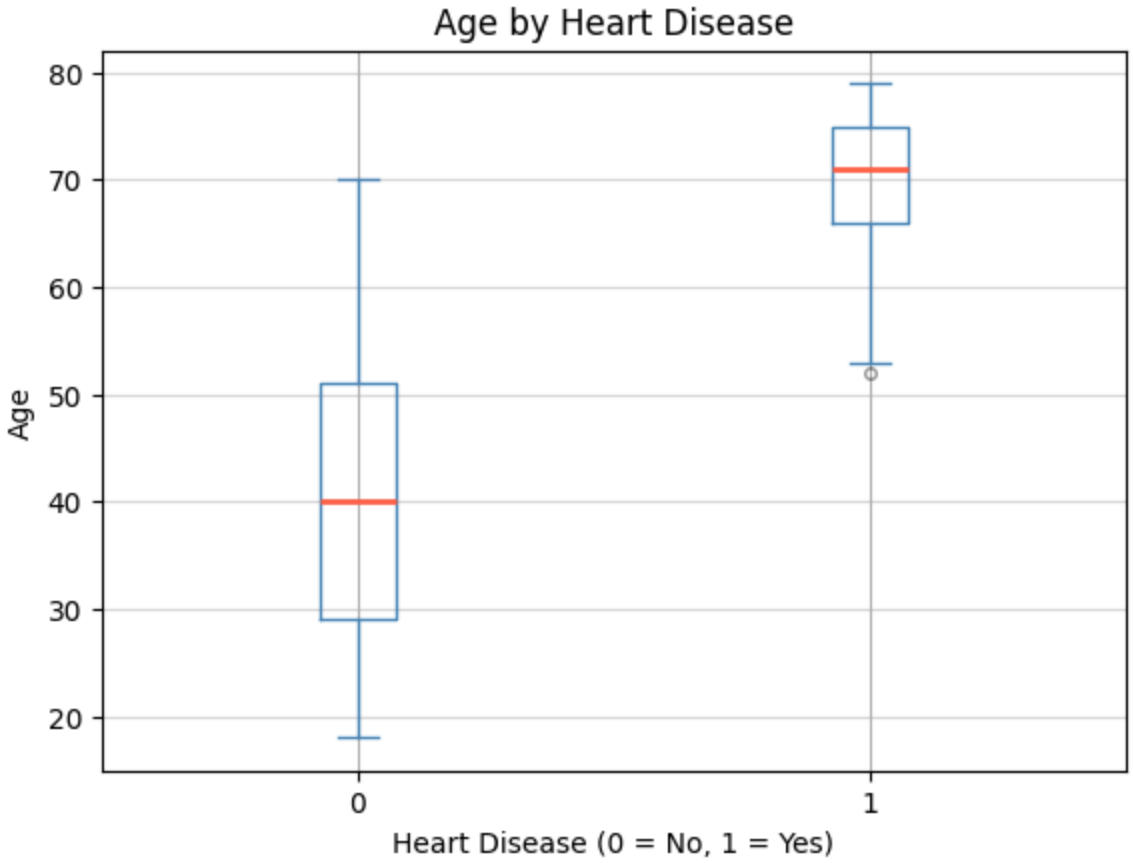
In [ ]:
```
data['diabetes'].value_counts().sort_index().plot(
    kind='bar',
    color='skyblue',
    edgecolor='black',
    alpha=0.7
)
plt.xlabel("Diabetes (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.title("Distribution of Diabetes")
plt.grid(axis='y', alpha=0.5)
plt.show()
```



Boxplots

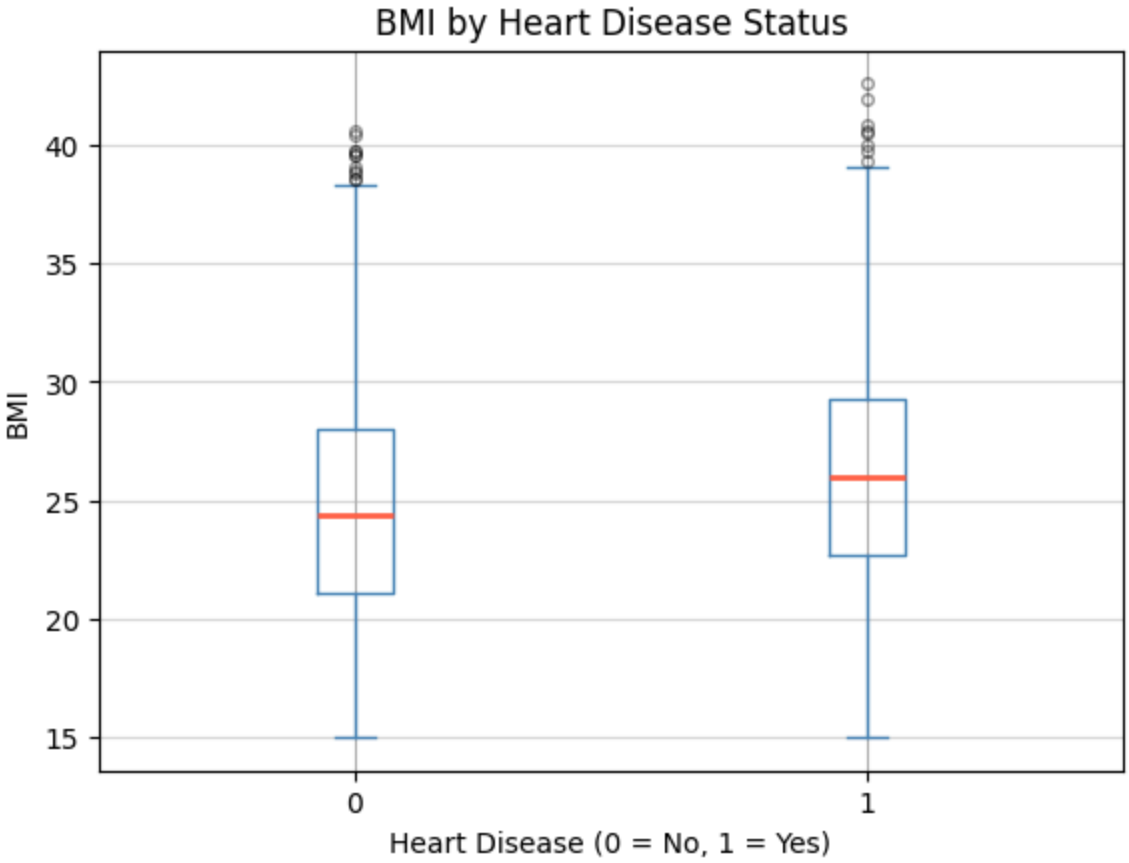
In [ ]:
```
data.boxplot(column='age', by='heart_disease',
    boxprops=dict(color='steelblue'),
    medianprops=dict(color='tomato', linewidth=2),
    whiskerprops=dict(color='steelblue'),
    capprops=dict(color='steelblue'),
    flierprops=dict(marker='o', markersize=4, alpha=0.4))

plt.title("Age by Heart Disease")
plt.suptitle("")
plt.xlabel("Heart Disease (0 = No, 1 = Yes)")
plt.ylabel("Age")
plt.grid(axis='y', alpha=0.5)
plt.show()
```

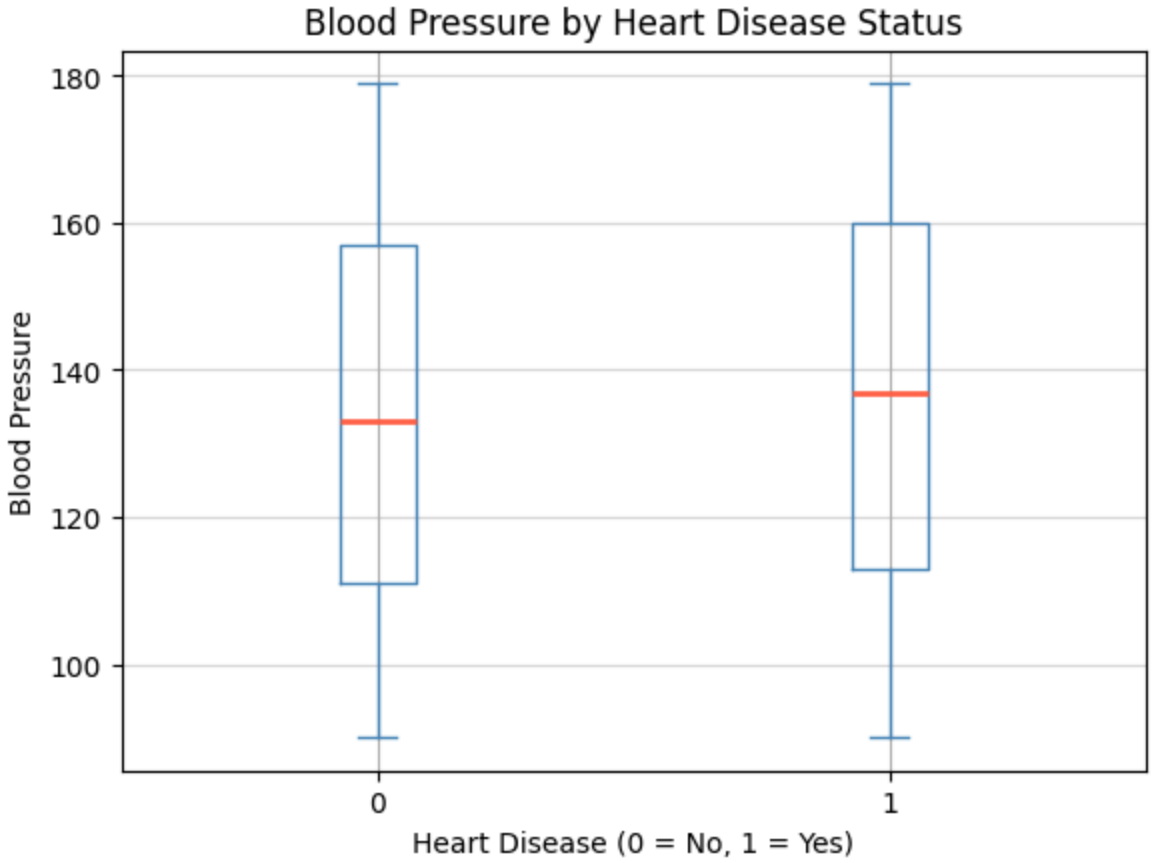


In [ ]:
```
data.boxplot(column='bmi', by='heart_disease',
             boxprops=dict(color='steelblue'),
             medianprops=dict(color='tomato', linewidth=2),
             whiskerprops=dict(color='steelblue'),
             capprops=dict(color='steelblue'),
             flierprops=dict(marker='o', alpha=0.4, markersize=4))

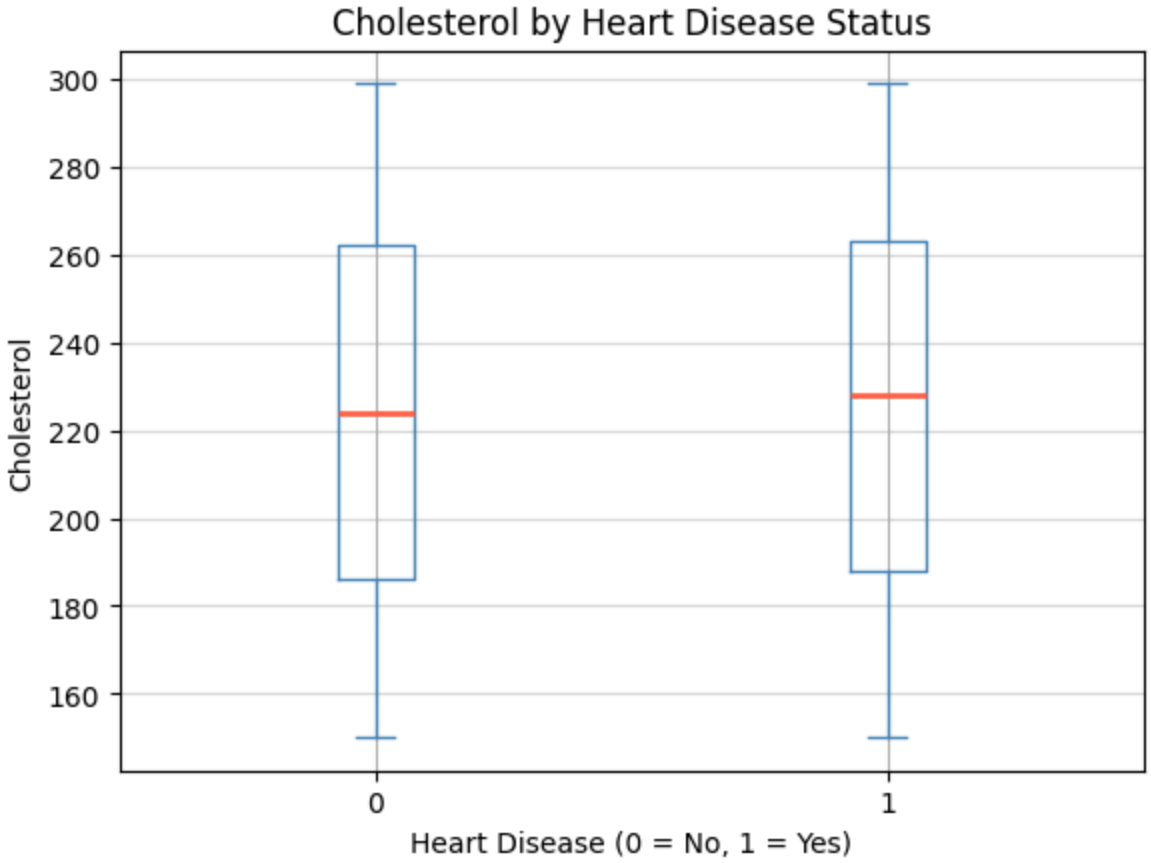
plt.title('BMI by Heart Disease Status')
plt.suptitle('')
plt.xlabel('Heart Disease (0 = No, 1 = Yes)')
plt.ylabel('BMI')
plt.grid(axis='y', alpha=0.5)
plt.show()
```



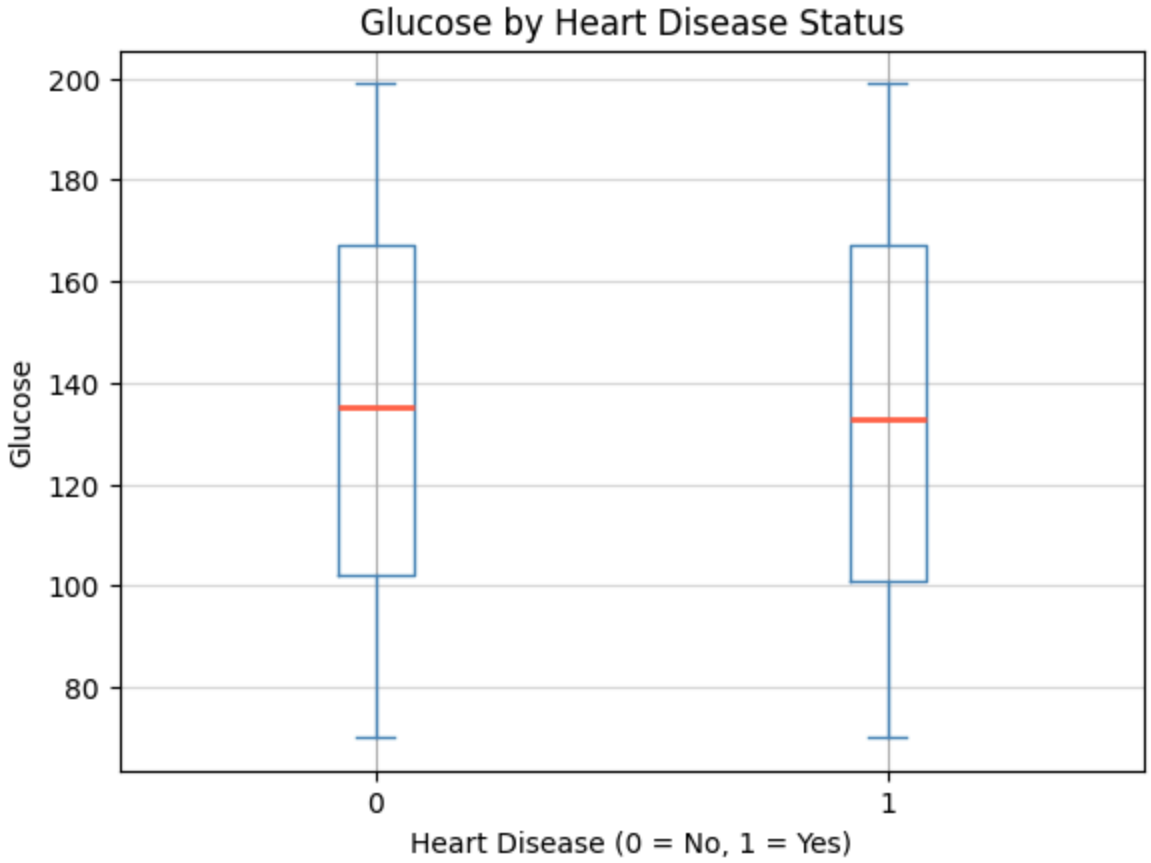
In [ ]:
```
data.boxplot(column='blood_pressure', by='heart_disease',
             boxprops=dict(color='steelblue'),
             medianprops=dict(color='tomato', linewidth=2),
             whiskerprops=dict(color='steelblue'),
             capprops=dict(color='steelblue'),
             flierprops=dict(marker='o', alpha=0.4, markersize=4))
plt.title('Blood Pressure by Heart Disease Status')
plt.suptitle('')
plt.xlabel('Heart Disease (0 = No, 1 = Yes)')
plt.ylabel('Blood Pressure')
plt.grid(axis='y', alpha=0.5)
plt.show()
```



In [ ]:
```
data.boxplot(column='cholesterol', by='heart_disease',
             boxprops=dict(color='steelblue'),
             medianprops=dict(color='tomato', linewidth=2),
             whiskerprops=dict(color='steelblue'),
             capprops=dict(color='steelblue'),
             flierprops=dict(marker='o', alpha=0.4, markersize=4))
plt.title('Cholesterol by Heart Disease Status')
plt.suptitle('')
plt.xlabel('Heart Disease (0 = No, 1 = Yes)')
plt.ylabel('Cholesterol')
plt.grid(axis='y', alpha=0.5)
plt.show()
```



In [ ]:
```
data.boxplot(column='glucose', by='heart_disease',
             boxprops=dict(color='steelblue'),
             medianprops=dict(color='tomato', linewidth=2),
             whiskerprops=dict(color='steelblue'),
             capprops=dict(color='steelblue'),
             flierprops=dict(marker='o', alpha=0.4, markersize=4))
plt.title('Glucose by Heart Disease Status')
plt.suptitle('')
plt.xlabel('Heart Disease (0 = No, 1 = Yes)')
plt.ylabel('Glucose')
plt.grid(axis='y', alpha=0.5)
plt.show()
```



Additional Visualizations

Pairplot

In [ ]:
```
import matplotlib.patches as mpatches

pairplot_features = ['age', 'bmi', 'blood_pressure', 'cholesterol', 'glucose

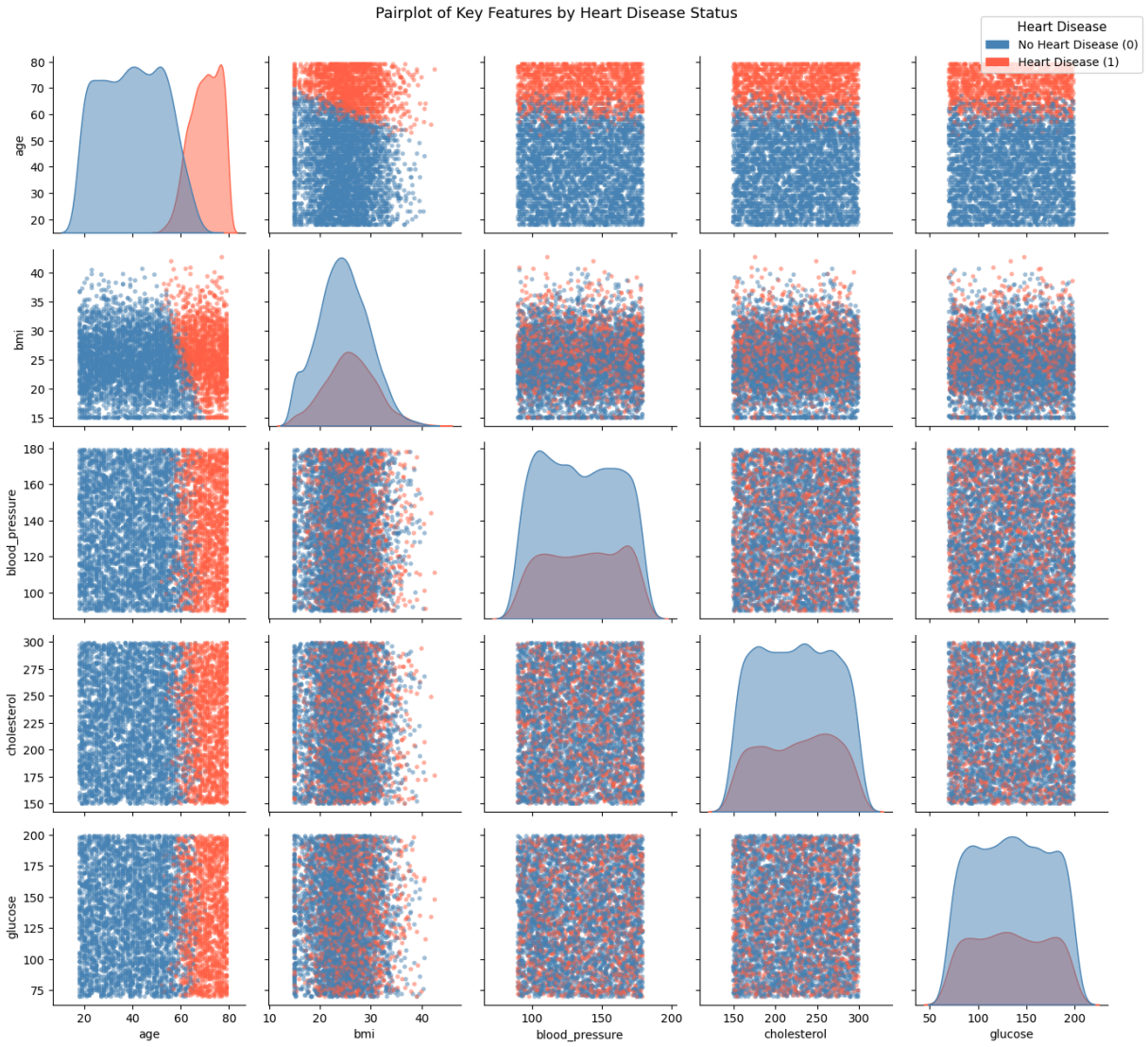
pp = sns.pairplot(
    data[pairplot_features],
    hue='heart_disease',
    palette={0: 'steelblue', 1: 'tomato'},
    diag_kind='kde',
    plot_kws={'alpha': 0.5, 'edgecolor': 'none', 's': 15},
    diag_kws={'fill': True, 'alpha': 0.5}
)

no_disease  = mpatches.Patch(color='steelblue', label='No Heart Disease (0)'
yes_disease = mpatches.Patch(color='tomato',    label='Heart Disease (1)')

pp.figure.legend(
    handles=[no_disease, yes_disease],
    title='Heart Disease',
    loc='upper right',
    bbox_to_anchor=(1.02, 1),
    borderaxespad=0,
    frameon=True,
    fontsize=10,
```

```
    title_fontsize=11
)
pp.legend.remove()

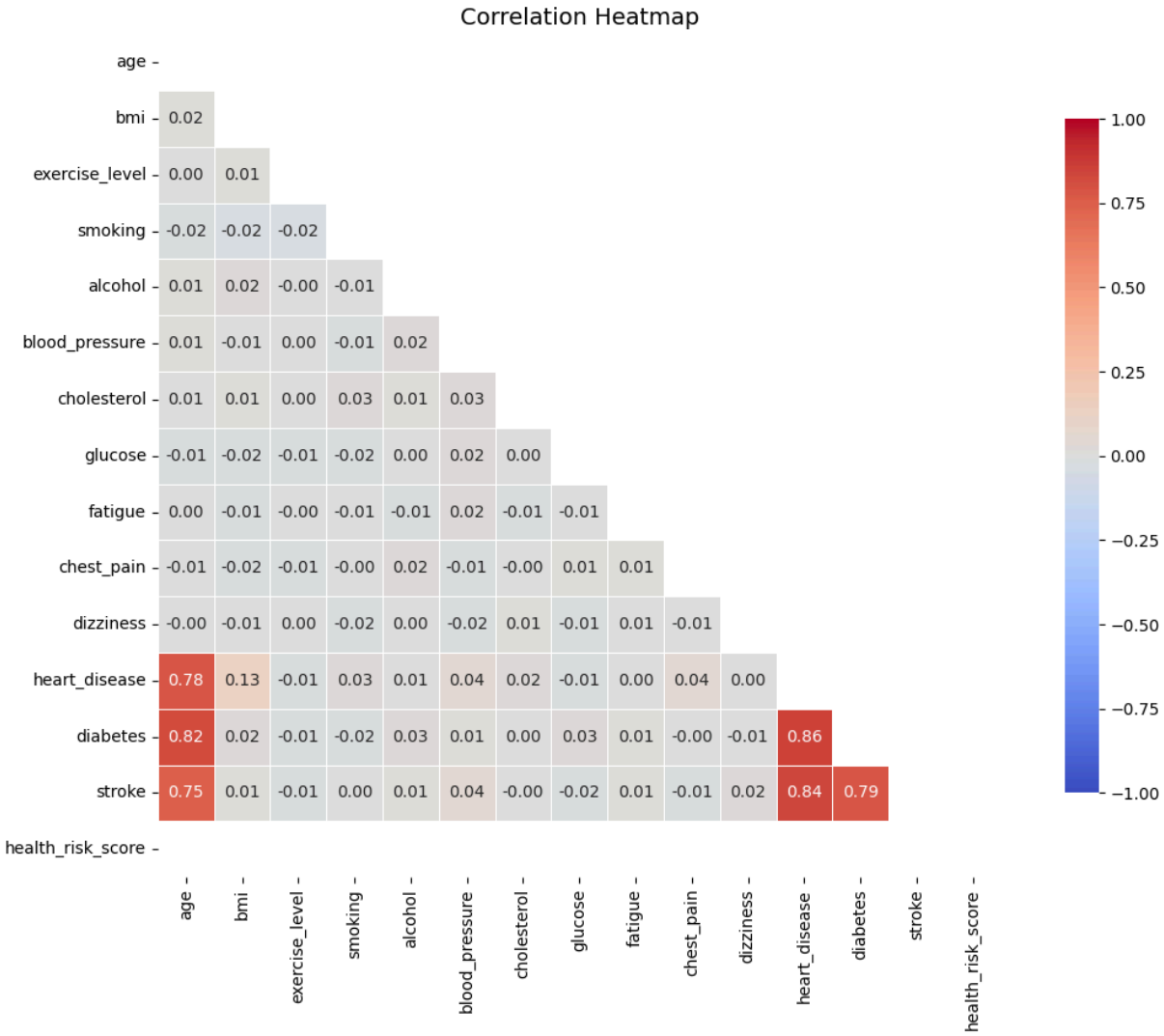

plt.suptitle('Pairplot of Key Features by Heart Disease Status', y=1.01, for
plt.tight_layout()
plt.show()
```



Correlation Heatmap

In [ ]:
```
plt.figure(figsize=(12, 9))
corr_matrix = data.select_dtypes(include=np.number).corr()
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f',
            cmap='coolwarm', center=0, vmin=-1, vmax=1,
            linewidths=0.5, square=True, cbar_kws={'shrink': 0.8})
plt.title('Correlation Heatmap', fontsize=14)
plt.tight_layout()
plt.show()
```



Correlation Analysis

In [ ]:
```
# Select only numeric columns from the DataFrame
numeric_data = data.select_dtypes(include=np.number)

# Correlation of all numeric columns with 'heart_disease'
correlation = numeric_data.corr()['heart_disease']
correlation
```

Out[ ]:

	heart_disease
age	0.776642
bmi	0.133455
exercise_level	-0.010010
smoking	0.027161
alcohol	0.007335
blood_pressure	0.041059
cholesterol	0.018736
glucose	-0.009114
fatigue	0.004118
chest_pain	0.038855
dizziness	0.001220
heart_disease	1.000000
diabetes	0.857576
stroke	0.842684
health_risk_score	NaN

dtype: float64

3. Data Quality and Cleaning

Data Quality Issues

Missing Values:

There are no records that contain an n/a, so none are missing.

Invalid Zeros:

A zero would be invalid if they exist in the following columns: gender (variable is categorical and either 'Male' or 'Female), bmi, blood pressure, cholesterol, glucose, and health_risk_score.

There are no unexpected zeros in this dataset.

Inconsistent Labels:

We expect the following options for each variable:

age - int 18 - 79

gender - 'Female', 'Male'

bmi - float 0 - 200

exercise_level - int 0, 1, 2

smoking - int 0,1

alcohol - int 0,1

blood_pressure - int 1 - 250

cholesterol - 1 - 500

glucose 1 - 500

fatigue - 0, 1

chest_pain - 0,1

dizziness - 0,1

diabetes - 0,1

stroke - 0,1

heart_disease - 0,1

Analysis of the data shows that no records do not fall within these labelling expectations.

Duplicate Records:

Because there is no unique identifier for each entry, **there is no absolute way to determine if there are duplicate records.**

Clearly Invalid Entries:

A clearly invalid entry would be a record that is not physically possible given medical knowledge. Due to the diversity of medical circumstances, we need to be tolerant of many different compositions of the data (unexpected or unusual patient profiles), so unless the value is extreme or labelled incorrectly, we don't truly have an 'invalid' designation.

Therefore, there are **no clearly invalid entries in the dataset.**

Unusual or Extreme Values:

As mentioned before, we are highly tolerant of unusual values. However, some expectations can be made with medical knowledge. For example, a glucose level of 700 would be quite inconceivable. **None such inconceivable, extreme values medically were found.**

4. Modeling Readiness Reflection

Most Important Patterns

The exploratory analysis revealed that lifestyle and health-related features (such as BMI, smoking status, and physical activity) show noticeable relationships with the target variable, Heart_Disease. No single feature appears to dominate prediction, suggesting that a combination of variables will be needed for accurate modeling. Additionally, some features show moderate correlations with each other, indicating potential multicollinearity that may need to be considered in certain models.

Most Important Data Quality Issues

The dataset is largely clean with no missing values, which is a strong advantage. Outliers are only present when further analysis is done, such as the scatterplots with BMI by heart disease, but not the raw data itself.

Most Useful Features

Features related to direct health indicators and behaviors appear most useful, including BMI, blood pressure-related variables, and lifestyle factors such as smoking, alcohol use, and physical activity. These variables show clearer patterns in both visualizations and correlation analysis, suggesting they will provide meaningful predictive signal.

Preprocessing/Modeling Challenges

Key challenges moving forward include handling invalid zero values through imputation or filtering, and applying feature scaling for models sensitive to differences in magnitude. Potential class imbalance in the Heart_Disease variable may require resampling techniques or adjusted evaluation metrics. Additionally, model selection will involve balancing interpretability (such as logistic regression) with predictive performance (such as tree-based models), while also addressing any multicollinearity between features.